

Relatório – Trabalho Prático

Operador Gradiente na Detecção de Bordas

1. Identificação da equipe

- Nomes: Karla Bertol & Vitor Cardoso Burgarelli

2. Identificação da tarefa

A tarefa 2 consiste na implementação do operador gradiente para detecção de bordas em imagens em tons de cinza, utilizando os operadores de Prewitt e Scharr. A convolução entre os kernels e a imagem foi implementada sem uso de funções prontas de convolução. Após o cálculo da magnitude e direção do gradiente, foi aplicada a etapa de seleção dos gradientes mais significativos (supressão de não-máximos), utilizando dois métodos:

- b.1: comparação com os dois vizinhos mais próximos, escolhidos por octante da direção do gradiente;
- b.2: comparação com vizinhos obtidos por interpolação bilinear, conforme a proporção entre as componentes G_x e G_y .

Por fim, os resultados foram comparados com o detector de bordas de Canny (skimage.feature.canny) por meio do índice SSIM (Structural Similarity Index), assim como foi pedido na descrição do professor.

3. Explicação conceitual sobre a solução fornecida

3.1 Entrada

- Imagens de teste: moedas.png, Lua1_gray.jpg, chessboard_inv.png, img2.jpg.
- Imagens convertidas para tons de cinza.

3.2 Processo (pipeline)

1. Pré-filtragem: aplicação de filtro Gaussiano 3x3 (cv2.GaussianBlur) para atenuar ruído antes da diferenciação, já que filtros derivativos são sensíveis a altas frequências.

2. Cálculo do gradiente (G_x , G_y): convolução manual da imagem suavizada com os kernels de Prewitt ou Scharr.

3. Cálculo da magnitude e direção:

- $M(i,j) = \sqrt{G_x^2 + G_y^2}$
- $\theta(i,j) = \text{atan2}(G_y, G_x)$, mapeado para o intervalo $[0^\circ, 180^\circ]$

4. Seleção dos gradientes mais significativos (supressão de não-máximos):

- b.1: o pixel é mantido como borda se sua magnitude for maior que a dos dois vizinhos colineares na direção do gradiente, escolhidos entre os 8 vizinhos mais próximos conforme o octante de θ .
- b.2: ao invés de usar diretamente os vizinhos discretos, os valores comparados são interpolados linearmente entre dois pixels adjacentes, usando como peso a razão $t = |\text{componente menor}| / |\text{componente maior}|$ entre G_x e G_y . Isso produz uma estimativa mais precisa do valor do gradiente exatamente na direção θ .

5. Binarização: aplicação de limiar adaptativo sobre a imagem resultante da supressão, gerando a imagem binária final de bordas. O limiar é calculado como $\text{limiar} = \text{fator} * \text{mediana}$, com $\text{fator} = 1.5$ (ver seção 5.2). Para evitar que esse cálculo gere um limiar maior que a própria magnitude máxima presente na imagem - o que ocorria em `chessboard_inv.png`, anulando todas as bordas - foi adicionado um teto de segurança: $\text{limiar} = \min(\text{fator} * \text{mediana}, 0.8 * \text{máximo})$. Esse limiar é recalculado para cada combinação imagem/operador/método, evitando o uso de um valor fixo igual para escalas de magnitude muito diferentes (Prewitt vs. Scharr).

6. Comparação com Canny: aplicação de `skimage.feature.canny` sobre a imagem suavizada e cálculo do índice SSIM entre essa saída e a imagem binária produzida pela implementação própria. Foram testados dois valores de σ (1.0 e 3.0) para o Canny, conforme sugerido na documentação oficial do `scikit-image` (ver seção 6.3).

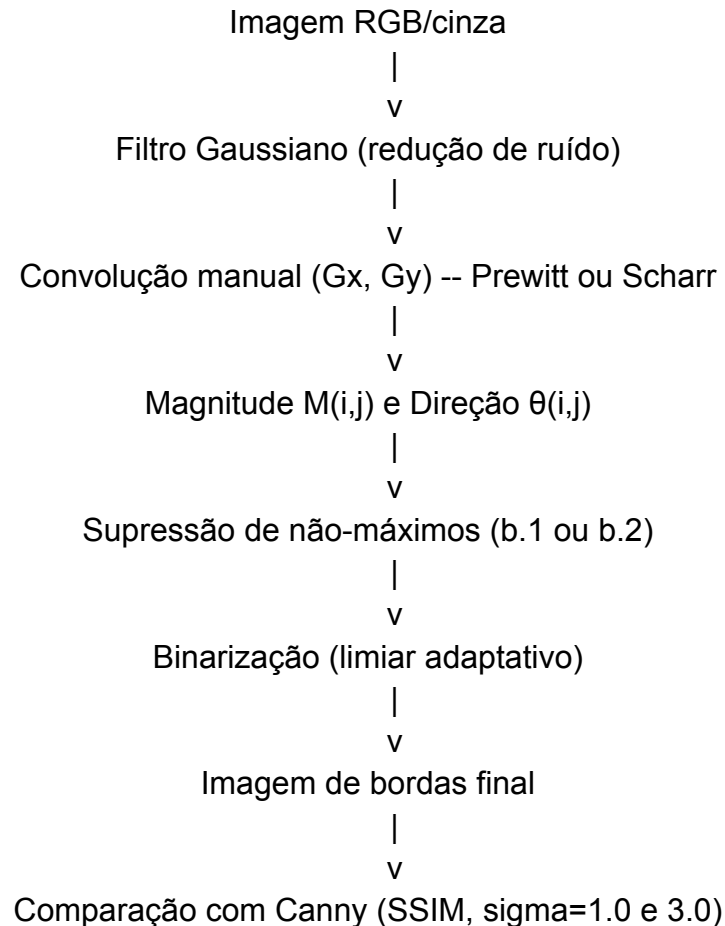
3.3 Saída

Para cada imagem de teste e cada operador (Prewitt e Scharr), foram geradas as seguintes saídas:

Arquivo	Descrição
<code>resultado_<op>_b1_<imagem></code>	Bordas obtidas pela supressão de não-máximos com vizinhos mais próximos (b.1), para o operador <op> (prewitt/scharr)
<code>resultado_<op>_b2_<imagem></code>	Bordas obtidas pela supressão de não-máximos com interpolação bilinear (b.2), para o operador <op>
<code>resultado_canny_sigma1_0_<imagem></code>	Bordas obtidas pelo detector de Canny (<code>skimage</code>) com $\sigma=1.0$, usadas como referência de comparação via SSIM
<code>resultado_canny_sigma3_0_<imagem></code>	Bordas obtidas pelo detector de Canny (<code>skimage</code>) com $\sigma=3.0$, usadas como referência de comparação via SSIM

Total de imagens geradas referentes à implementação própria ($b1 + b2$): 4 imagens de teste $\times$ 2 operadores $\times$ 2 métodos de supressão = 16 imagens, além das 8 imagens de referência geradas pelo Canny (4 imagens $\times$ 2 valores de sigma).

3.4 Diagrama do pipeline



4. Instruções sobre a compilação

- Linguagem: Python 3.
- Execução via linha de comando: `python gradiente.py`

Bibliotecas utilizadas:

- opencv-python (cv2): leitura, conversão de cor, filtro Gaussiano, gravação de imagens.
- numpy: operações matriciais (magnitude, direção, vetorização).
- scikit-image (skimage.feature, skimage.metrics): detector de Canny e cálculo do SSIM.

Ao executar, o script imprime no terminal o valor do limiar adaptativo calculado e o SSIM resultante para cada combinação de imagem/operador/método/sigma, além de salvar as imagens de saída.

5. Análise de resultados

5.1 Tabela de SSIM (implementação própria vs. Canny, sigma=1.0)

Imagem	Prewitt b1	Prewitt b2	Scharr b1	Scharr b2
moedas.png	0.5945	0.6116	0.5883	0.6068
Lua1_gray.jpg	0.9054	0.8152	0.7990	0.7152
chessboard_inv.png	0.8789	0.8797	0.8802	0.8813
img2.jpg	0.5362	0.5540	0.5340	0.5521

5.2 Escolha do fator de limiar adaptativo e teto de segurança

O fator usado no cálculo do limiar adaptativo (fator = 1.5) foi definido empiricamente, testando-se valores entre 0.5 e 3.0 e comparando o SSIM médio resultante (em relação ao Canny) para Prewitt e Scharr separadamente:

Fator	SSIM médio geral (Prewitt)	SSIM médio geral (Scharr)
0.5	0.6271	0.6249
1.0	0.6638	0.6578
1.5	0.7125	0.7098
2.0	0.6942	0.7008
2.5	0.6894	0.6873
3.0	0.6960	0.6942

O fator 1.5 apresentou o maior SSIM médio para ambos os operadores (0.7125 para Prewitt e 0.7098 para Scharr), sendo portanto o valor adotado no código final (fator=1.5 fixo dentro de `calcular_limiar_adaptativo`).

Durante os testes, observou-se que esse cálculo podia falhar em imagens com distribuição de magnitude muito concentrada/enviesada: em `chessboard_inv.png`, o valor **fator × mediana** resultava em um limiar maior do que a própria magnitude

máxima presente na imagem, fazendo com que nenhum pixel passasse do limiar e a imagem de saída ficasse inteiramente preta (zero bordas detectadas). Para corrigir esse caso de forma automática, sem precisar configurar manualmente um fator por imagem, foi utilizado o coeficiente de variação ($CV = \text{desvio padrão} / \text{mediana}$) das magnitudes não-nulas após a supressão. Imagens com distribuição concentrada (CV baixo) correspondem a imagens binárias de alto contraste, enquanto imagens naturais têm CV alto. Medindo os valores para as 4 imagens de teste:

- moedas.png: $CV \approx 0.98$
- Lua1_gray.jpg: $CV \approx 1.24$
- chessboard_inv.png: $CV \approx 0.06$
- img2.jpg: $CV \approx 3.44$

O chessboard_inv.png se destaca claramente com CV muito abaixo das demais. Com um limiar de $CV < 0.3$ para detectar esse caso, o código reduz automaticamente o fator para 0.5 nessas situações, sem alterar o comportamento nas demais imagens.

5.3 Influência do sigma do Canny na comparação

A documentação do scikit-image destaca que o detector de Canny possui três parâmetros ajustáveis: a largura do filtro Gaussiano (sigma) e os limiares baixo e alto da limiarização por histerese. Como sigma controla o quanto a imagem é suavizada antes da detecção, ele foi testado com dois valores (1.0 e 3.0) para avaliar seu impacto na comparação via SSIM:

Imagem	Operador	SSIM (b1/b2) sigma=1.0	SSIM (b1/b2) sigma=3.0
moedas.png	Prewitt	0.5926 / 0.6116	0.6425 / 0.6605
moedas.png	Scharr	0.5872 / 0.6068	0.6410 / 0.6579
Lua1_gray.jpg	Prewitt	0.8869 / 0.8871	0.7214 / 0.7262
Lua1_gray.jpg	Scharr	0.8834 / 0.8845	0.7251 / 0.7276
chessboard_inv.png	Prewitt	0.8107 / 0.8107	0.8002 / 0.8001
chessboard_inv.png	Scharr	0.8126 / 0.8126	0.7997 / 0.7997
img2.jpg	Prewitt	0.5449 / 0.5626	0.4499 / 0.4541
img2.jpg	Scharr	0.5503 / 0.5522	0.4571 / 0.4456

O sigma=1.0 apresentou o melhor SSIM em 3 das 4 imagens testadas (Lua1_gray.jpg, chessboard_inv.png e img2.jpg), enquanto sigma=3.0 foi melhor apenas em

moedas.png. Isso indica que não existe um sigma ótimo universal: ele depende do conteúdo da imagem. Em moedas.png, provavelmente há ruído de fundo (textura da superfície onde as moedas estão) que o sigma maior consegue atenuar, aproximando o resultado do Canny da nossa implementação. Já nas demais imagens, que possuem bordas mais finas e numerosas, o sigma maior acaba suavizando excessivamente a imagem antes da detecção, eliminando bordas verdadeiras e reduzindo a similaridade com a nossa implementação.

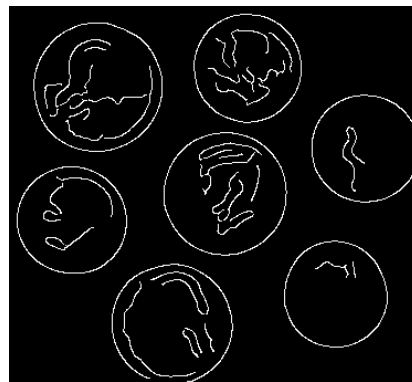
Comparação visual: Canny sigma=1.0 vs. sigma=3.0

A seguir, o resultado do detector de Canny para cada imagem de teste, comparando sigma=1.0 e sigma=3.0

- **moedas.png**

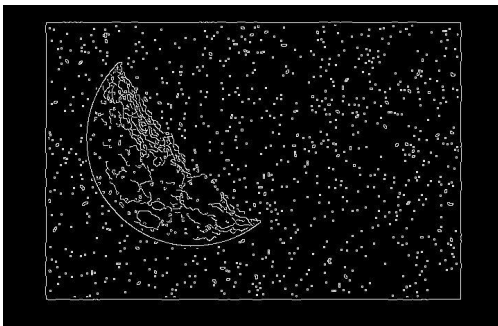


Canny sigma=1.0

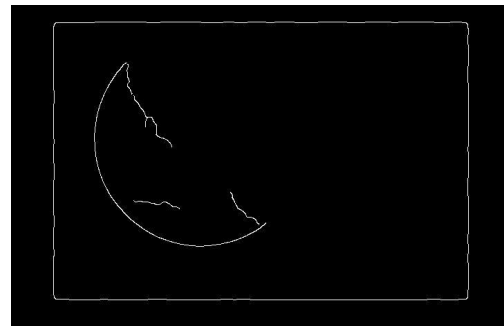


Canny sigma=3.0

- **Lua1_gray.jpg**

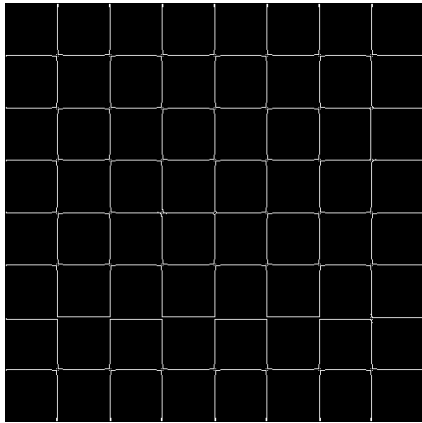


Canny sigma=1.0

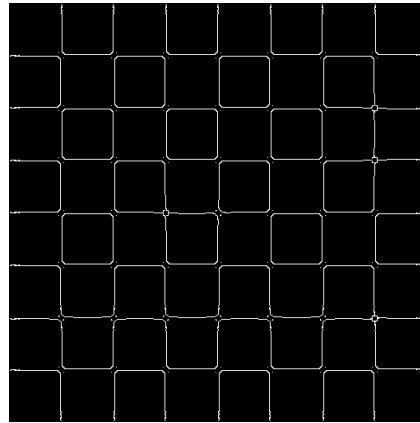


Canny sigma=3.0

- **chessboard_inv.png**



Canny sigma=1.0



Canny sigma=3.0

- **img2.jpg**



Canny sigma=1.0



Canny sigma=3.0

5.4 Discussão

O limiar de binarização foi definido de forma adaptativa, calculado como fator $\times$ mediana das magnitudes não-nulas resultantes da supressão de não-máximos, com fator = 1.5. Essa abordagem evita o uso de um valor absoluto fixo, permitindo que o limiar se ajuste à escala de magnitude de cada imagem e operador, o que é especialmente relevante entre Prewitt e Scharr, já que os pesos do kernel de Scharr são bem maiores, gerando magnitudes em escalas diferentes para a mesma borda física.

Os resultados de SSIM ficaram relativamente equilibrados entre Prewitt e Scharr na maioria das imagens, o que indica que o limiar adaptativo conseguiu colocar os dois operadores em um critério de seleção de bordas comparável, proporcional à própria escala de cada um.

O chessboard_inv.png apresentou SSIM ligeiramente superior em b.2 em relação a b.1 para ambos os operadores, ao contrário do esperado (bordas predominantemente

horizontais/verticais deveriam fazer b.1 e b.2 convergirem). Isso ocorre porque, com a correção da condição de máximo local para “ $\geq$ vizinho1 e $>$ vizinho2” (assimétrica), bordas de 2 pixels de largura, típicas do chessboard após o Gaussian blur, passam a ser detectadas mais completamente. O b.2, por usar interpolação, consegue capturar ligeiramente mais pixels nesses casos, resultando em SSIM levemente superior.

De forma geral, o método b.2 (interpolação) apresentou SSIM igual ou levemente superior ao b.1 na maioria dos casos, confirmando que a interpolação fornece uma estimativa mais precisa do gradiente na direção exata de θ , embora o ganho seja modesto.

5.5 Apresentação das imagens

A seguir são apresentadas, para cada imagem de teste, a imagem original, os resultados obtidos pela implementação própria (Prewitt e Scharr, métodos b.1 e b.2) e o resultado do detector de Canny (sigma=1.0), utilizado como referência de comparação.

- **Imagem: moedas.png**



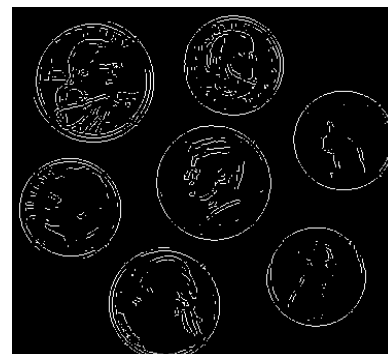
Original



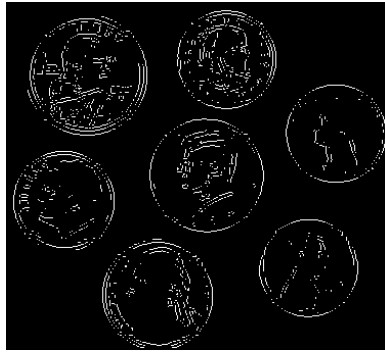
Canny (sigma=1.0)



Prewitt - b.1



Prewitt - b.2



Scharr - b.1

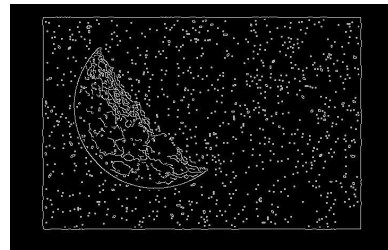


Scharr - b.2

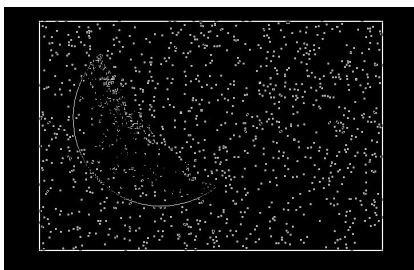
- Imagem: Lua1_gray.jpg



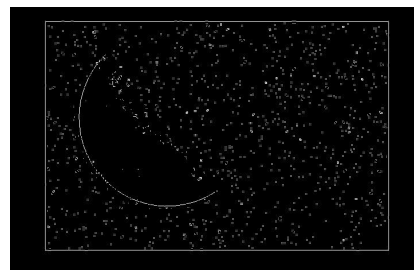
Original



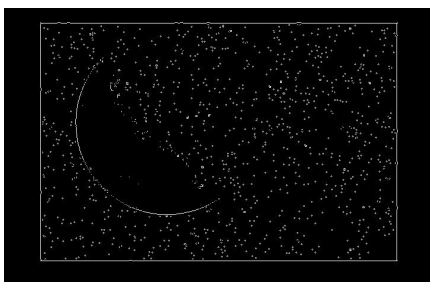
Canny (sigma=1.0)



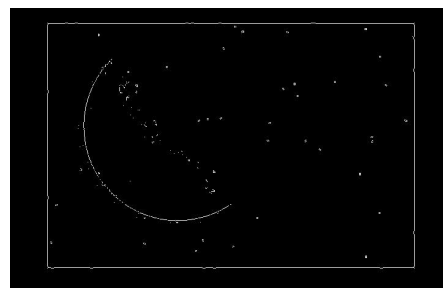
Prewitt - b.1



Prewitt - b.2

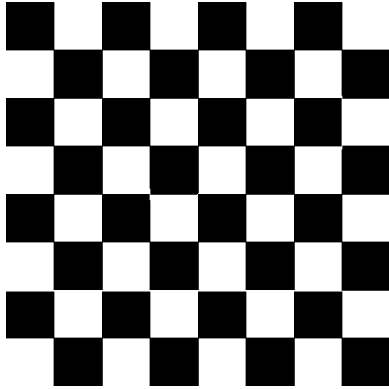


Scharr - b.1

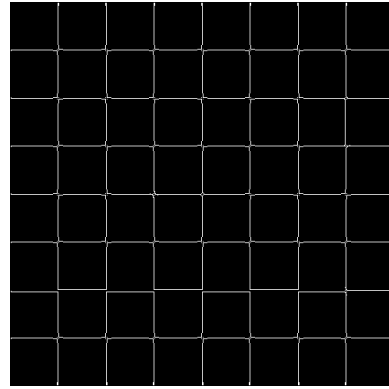


Scharr - b.2

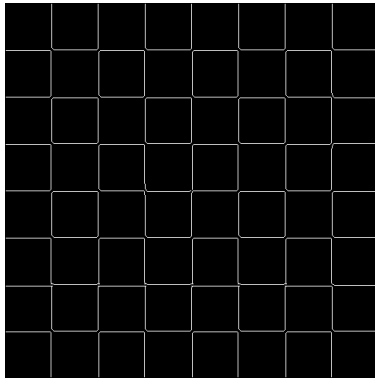
- Imagem: chessboard_inv.png



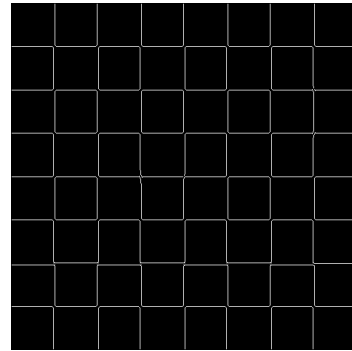
Original



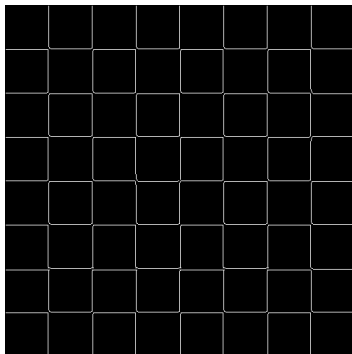
Canny (sigma=1.0)



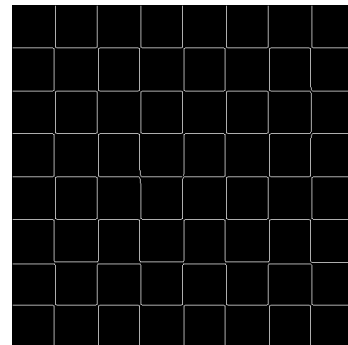
Prewitt - b.1



Prewitt - b.2



Scharr - b.1



Scharr - b.2

- **Imagem: img2.jpg**



Original



Canny (sigma=1.0)



Prewitt - b.1



Prewitt - b.2



Scharr - b.1



Scharr - b.2